

CSC483/583 Programming Project: Three-Stage Wikipedia Clue Answering

Cole Mobberley, Suhani Surana, Aunabil Chakma
University of Arizona

Abstract

In this project, we study Wikipedia clue answering as a retrieval problem: each clue must be mapped to the title of a Wikipedia page. The dataset contains 100 J-Archive clues whose gold answers are Wikipedia titles, and the document collection contains 280,000 article pages from a parsed Wikipedia dump. Our system uses a three-stage pipeline. Stage 1 builds a broad candidate list with weighted sparse and dense retrieval. Stage 2 reranks the top 100 candidates with a cross-encoder using natural-question reformulations of the clue. Stage 3 reranks only the top 10 candidates with Qwen3-Reranker-4B. Our system achieves a P@1 score of 0.47 on these clues, showing that selection improves when broad retrieval is followed by stronger rerankers over much smaller candidate sets. Github Link: github.com/aunabilChakmaArizona/csc-483-583-team-project

1 Introduction

Wikipedia clue answering is difficult because clues are often indirect, category-dependent, and phrased differently from the text that appears in the target article. Rather than asking for a direct fact, many clues refer to an entity through paraphrase, role, quoted language, or historical description. This creates a lexical mismatch problem: the words in the clue are often not the words that appear most prominently in the correct Wikipedia page.

We treat the task as a page retrieval over Wikipedia. Given a category and a clue, the goal is to predict the Wikipedia page title that matches the gold answer. This approach avoids full generative question answering and turns the task into candidate generation and re-ranking over 280,000 articles. The main challenge is retrieving the correct page despite noisy Wikipedia markup, long article bodies, redirect pages, and clues whose phrasing differs sharply from natural encyclopedia prose.

Our core design principle is to use simpler and cheaper methods early, and then apply larger semantic models later, only to narrow candidate sets. The resulting system is a three-stage retrieval pipeline (See Figure 1). Stage 1 uses weighted hybrid retrieval, which combines BM25 ranking over cleaned article text with Dense Passage Retrieval (DPR) embeddings [Karpukhin et al., 2020] indexed in FAISS [Johnson et al., 2019]. Stage 2 reranks the top 100 candidates by combining the Stage 1 score with a cross-encoder score from a model trained for passage ranking [Reimers and Gurevych, 2019]. Stage 3 exports only the top 10 candidates from Stage 2 and reranks them with Qwen3-Reranker-4B, a larger reranker model [Qwen Team, 2025]. We also reformulate each clue into five natural-language questions so that the dense retriever and rerankers receive inputs that better match question-versus-passage training distributions. Throughout the paper, Precision@1 (P@1) is the main evaluation metric because the system must return one final Wikipedia title as its answer.

This staged architecture yields consistent gains across the pipeline. Stage 1 reaches a P@1 score of 0.31, Stage 2 improves this to 0.41, and Stage 3 reaches 0.47. These results support a simple conclusion:

retrieval quality matters first, but stronger semantic reranking becomes effective once the correct page has been brought into a small candidate set.

2 Task and Dataset

Our task is to answer clue-style questions by retrieving the Wikipedia page whose title matches the gold answer. The clues come from J-Archive and follow the Jeopardy-style format, but the system output is not free text: each test example contains a category, a clue, and one or two correct Wikipedia titles. This makes the problem a page-ranking task rather than a fully generative question-answering task.

We use the given test dataset for the project, containing 100 clues whose answers appear as Wikipedia page titles, together with a parsed Wikipedia collection of approximately 280,000 pages. After separating redirects from ordinary articles, we mainly do indexing and work on 152,042 non-redirect article pages. This setup creates two main challenges. First, the collection is large enough that retrieval quality matters immediately. Second, the clues are often indirect and category-dependent, so the wording of the clue may differ substantially from the wording used in the correct Wikipedia article.

3 Core Implementation

3.1 Wikipedia Preprocessing

We index the collection with Whoosh, treating each Wikipedia page as a document. We store parsed pages in SQLite, keep redirect pages separate from article pages, and build the retrieval pipeline over the article records. In the final system, we index only the non-redirect article pages, resulting in 152,042 indexed documents out of the roughly 280,000 parsed pages (Section 4; `file:src/processor1_parse.py`, `method: materialize_articles_to_sqlite()`).

We also prepared the text specifically for Wikipedia content. In the cleaning stage, we removed redirect lines, section headers, template markup, reference markup, file/image markup, and category lines, then normalized whitespace (Section 4; `file:src/processor2_clean.py`, `method: clean_body()`). Redirect pages were handled explicitly instead of being treated as normal article bodies because they do not contain meaningful page content and should not be indexed as ordinary evidence-bearing articles in the final system. Instead, the redirect plan was to index redirect titles as aliases and, when a clue matched a redirect title, transfer that score as a boost to the resolved canonical article page (Section 4; `file:src/processor_redirect.py`, `method: resolve_redirect()`).

For term preparation, we tried two approaches. The first was a custom normalization pipeline built in our preprocessing code: lowercasing, whitespace tokenization, boundary-punctuation stripping, stop-word removal using spaCy’s English stop-word list, removal of non-word characters, and splitting on underscores and hyphens (Section 4; `file:src/processor3_tokenize.py`, `method: tokenize_body()`). The second approach was to index the cleaned article text directly with Whoosh’s default analyzer, which applies its own tokenization, lowercasing, stop-word filtering, and stemming (Section 4; `file:src/processor4_cole_index.py`, `method: materialize_whoosh_cole_index()`). In our experiments, the default Whoosh analyzer performed better, so the final system uses it instead of the custom tokenization.

3.2 Indexing

We create multiple indexes (inverted), either using direct terms or embeddings, so that different evidence sources can be tested inside one weighted retrieval framework. The sparse (terms) indexes are created using Whoosh, while the dense (embeddings) representations (using DPR) are indexed with FAISS (Section 4;

Table 1: Retrieval representations used in the experiments. The first column names the text representation, the second column gives the index backend, and the last column describes what text or embeddings are searched. Except for the body-only baseline, sparse indexes use Whoosh’s default analyzer.

Representation	Index	Description
Body text	Whoosh	Sparse index over article body text processed with custom tokenization and normalization.
Title and body text	Whoosh	Sparse index over article titles together with article body text.
Redirect aliases	Whoosh	Sparse index over redirect-page titles, used as alternative names for canonical article pages.
First sentence	Whoosh	Sparse index over only the first sentence of each article.
First two sentences	Whoosh	Sparse index over only the first two lead sentences of each article.
Opening paragraph	Whoosh	Sparse index over only the opening paragraph of each article.
Article categories	Whoosh	Sparse index over Wikipedia category labels linked to each article.
Title with first sentence	FAISS	Dense index using DPR embeddings of the article title concatenated first sentence.
Title with first two sentences	FAISS	Dense index using DPR embeddings of the article title concatenated first two sentences.
Title with opening paragraph	FAISS	Dense index using DPR embeddings of the article title concatenated opening paragraph.
Title with full article	FAISS	Dense index using DPR embeddings of the article title concatenated full cleaned article text.
Body summary	FAISS	Dense index using DPR embeddings of article-body summaries generated with Qwen.

file:src/prepare_dpr_articles.py, method: export_dpr_articles() file: src/colab_build_dpr_faiss.py, method: build_variant_index()). Table 1 summarizes the index variants we built.

In addition to these indexes, we implemented two simple pattern-based bonuses: one checks whether a four-digit year from the clue appears in the candidate article, and the other checks whether quoted text from the clue appears in the candidate article (Section 4; file:src/search.py, method: build_year_match_query(), build_quote_match_query()). We also tested category-semantic variants, where Wikipedia category labels linked to an article are appended to the dense text representation so that retrieval can use topic labels in addition to article body text (Section 4; file:src/processor_category_semantic.py, method: materialize_category_semantic_matches()). Because the clues are often fragments rather than natural questions, we also rewrote each clue into five natural-language questions with Qwen3-8B. These generated questions were used as dense-retrieval queries in Stage 1 and as inputs to later reranking stages (Section 4; file: src/colab_generate_natural_questions_qwen.py, method: generate_questions_for_item(), file: src/processor_natural_question_dpr_embeddings.py, method: materialize_natural_question_dpr_embeddings()). Appendix Figure 2 lists the text templates for inputs and prompts used for our system.

3.3 Stage 1: Weighted Hybrid Retrieval

In Stage 1, we build an initial ranking over the full Wikipedia collection using the category text with the clue text as the query. This stage combines one sparse retrieval score with three dense retrieval scores and is responsible for broad candidate generation before more expensive rerankers are applied (Section 4; file:src/run_test_100.py, method: search_questions() file:src/search.py, method: multi_search_whoosh_weighted_cole()).

For the final selected run, the active Stage 1 components are:

- BM25 retrieval on “Title and body” index for the category+clue query (weight 15.00)
- DPR retrieval for the category+clue query (weight 1.00)
- DPR retrieval averaged over five generated natural questions of clues (weight 1.00)
- DPR retrieval of the concatenation of those five generated natural questions (weight 1.00)

Each DPR retrieval score is computed over four dense document indexes: title with first sentence, title with first two sentences, title with first paragraph, and title with full article text. For a given query, we keep the maximum score returned by these four DPR indexes for each candidate page (Section 4; file:src/search.py, method: search_dpr_faiss()). In the natural-question mode, we first run DPR separately for each of the five generated questions and then average those candidate scores (Section 4; file:src/search.py, method: build_faiss_precomputed_components(), average_dense_component_results()). We retained title, redirect, lead-only, body-summary, year-based matching, quoted-text matching, and category-semantic dense variants in the codebase, but all received zero weight in the final configuration.

The final Stage 1 score is assembled after each active component has produced a candidate list. Sparse Whoosh scores are serialized, normalized by their own maximum score, and then merged with dense FAISS scores that are normalized to the unit interval (Section 4; file:src/search.py, method: serialize_results(), normalize_results_to_unit_interval(), normalize_dense_results_to_unit_interval()). The deeper ranking method then creates one row per candidate page, fills in zero scores for missing components, applies the selected weights, transfers redirect-title matches to the resolved article when possible, and sorts by total weighted score with component-level tie-breakers (Section 4; file:src/search.py, method: search_whoosh_weighted_with_searcher(), weighted_result_key()).

This stage produces a strong but still inexpensive global ranking. It searches the entire collection, surfaces the top candidates for each clue, and supplies the top 100 pages to the cross-encoder stage.

3.4 Stage 2: Cross-Encoder Reranking

The second stage reranks the top 100 candidates from the weighted hybrid retriever using a cross-encoder model, cross-encoder/ms-marco-MiniLM-L-12-v2 (33M). Each candidate article is represented by its title and cleaned article body, split into 100-token chunks. For each of the five generated natural questions, the system scores all chunks and keeps the best chunk score for that question. The final cross-encoder score for each candidate is the average across the five natural questions (Section 4; file:src/run_test_100.py, method: cross_encoder_query_texts(), cross_encoder_document_text(), chunk_text_by_whitespace_tokens(), score_cross_encoder_pairs(), rerank_results_with_cross_encoder()). The exact query and document text formats are shown in Appendix Figure 2.

We then combine the cross-encoder score with the initial Stage 1 score. Both scores are normalized within the top-100 candidate set, and the selected combination uses a weight of 1.00 for the initial score and 7.00 for the cross-encoder score. This combined ranking forms Stage 2 (Section 4; file:src/run_test_100.py, method: min_max_normalize_values(), fuse_initial_and_cross_encoder_results(), build_stage2_searches()). It remains relatively efficient because it only processes 100 candidates per clue, but it is much more semantically expressive than Stage 1.

3.5 Stage 3: Qwen3-Reranker-4B

Now that we have the top 10 candidates from Stage 2, in Stage 3, for each clue, we export the top 10 candidates along with the category, clue, gold answer, five generated natural questions, and a truncated evidence snippet from each candidate article (Section 4; file:src/export_qwen_verifier_inputs.py, method: candidate_key()). These exported rows are then scored with Qwen3-Reranker-4B.

We also tested a separate Qwen3-4B chat-model verifier that used a direct yes/no prompt over the same evidence snippets. This approach was not effective as a final ranking method, whereas Qwen3-Reranker-4B, a dedicated reranker, performs much better in the Stage 3 reranking setting (Section 6.1 file:src/colab_score_qwen_verifier_inputs.py, method: reranker_text(), process_reranker_inputs(), reranker_probabilities(), score_reranker_row()). The exact chat-verifier prompt and

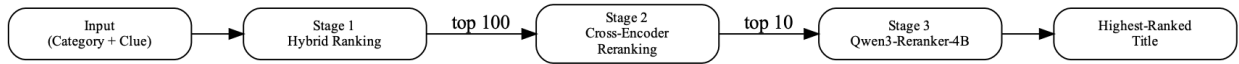


Figure 1: Final three-stage pipeline. The input category and clue are first mapped to a Stage 1 weighted hybrid ranking over Wikipedia pages. Stage 2 reranks the top 100 pages by combining the Stage 1 score with a cross-encoder score. Stage 3 reranks the top 10 Stage 2 pages with Qwen3-Reranker-4B and returns the highest-ranked title.

reranker input structure are listed in Appendix Figure 2. Stage 3 cannot recover a correct answer that was not already present in the exported top 10, so it is a pure answer-selection stage rather than a recall improvement stage.

4 Coding Documentation

The code can be run in a conda environment:

```
conda create -n wiki-clue-ir python=3.11
conda activate wiki-clue-ir
pip install -r requirements.txt
```

The final run expects the local processed data and index folders from the project run to be present. The source files are organized as follows:

- `src/processor1_parse.py`
 - `materialize_articles_to_sqlite()`
 - for `article` in `iter_articles(raw_dir)`:
 - `batch.append(article_row(article))`
 - if `len(batch) >= batch_size`:
 - `write_batch(connection, batch)`
 - `total += len(batch)`
 - `batch = []`

This method converts the raw Wikipedia dump into page-level SQLite records. It is where the project changes from file-level text to article-level documents.

- `src/processor2_clean.py`
 - `clean_body()`
 - `cleaned = remove_redirect_lines(cleaned)`
 - `cleaned = remove_section_headers(cleaned)`
 - `cleaned = remove_templates(cleaned)`
 - `cleaned = remove_references(cleaned)`
 - `cleaned = remove_file_markup(cleaned)`

This method removes Wikipedia markup that should not be treated as article evidence. The output is the cleaned body text used by the sparse index, dense export, and reranking stages.

- `src/processor4_cole_index.py`
 - `materialize_whoosh_cole_index()`

```

if article.is_redirect:
    skipped_redirects += 1
    continue
writer.add_document(title=article.title, body=article.body, ...)

```

This builds the final sparse Whoosh index over non-redirect article bodies. It is the positive-weight BM25 component in Stage 1.

- `src/prepare_dpr_articles.py`

```

- prepare_article_record()

return DprArticleRecord(
    title=article.title,
    first_sentence_text=get_first_sentence(article.body),
    first_two_sentences_text=get_first_two_sentences(article.body),
    first_paragraph_text=get_first_paragraph(article.body),
)

```

This method prepares the article text variants used for DPR embeddings. Those variants let the dense retriever compare a clue against different amounts of article context.

- `src/colab_build_dpr_faiss.py`

```

- build_variant_index()

embeddings = np.concatenate(embedding_batches, axis=0)
index = faiss.IndexFlatIP(embeddings.shape[1])
index.add(embeddings)
faiss.write_index(index, str(variant_dir / "index.faiss"))

```

This method encodes article variants with the DPR context encoder and stores them in FAISS. It creates the dense indexes searched by Stage 1.

- `src/colab_generate_natural_questions_qwen.py`

```

- generate_questions_for_item()

messages = build_messages(category, clue)
outputs = model.generate(**inputs, max_new_tokens=max_new_tokens)
questions = parse_questions_from_text(decoded_text)

```

This method rewrites each clue into five natural questions. These generated questions are later used as DPR queries and as reranker inputs.

- `src/processor_natural_question_dpr_embeddings.py`

```

- materialize_natural_question_dpr_embeddings()

flat_embeddings = encode_question_texts(flat_question_texts, ...)
combined_embeddings = encode_question_texts(combined_texts, ...)
question_embeddings = flat_embeddings.reshape(...)
np.savez_compressed(
    output_path,
    natural_questions_embeddings=question_embeddings,
)

```

This method precomputes DPR embeddings for both individual generated questions and their concatenation. Precomputing keeps the main evaluation run from repeatedly encoding the same queries.

- `src/search.py`

```

- multi_search_whoosh_weighted_cole()

all_results.append({
    "query": query_text,
    "results": search_whoosh_weighted_with_searcher(
        searcher, query_text=query_text, body_weight=body_weight, ...
    ),
})

```

This method is the Stage 1 batch search entry point. It applies the final weighted hybrid retrieval setup over sparse Whoosh scores and DPR/FAISS scores.

```
- search_dpr_faiss()

scores, row_indexes = current_index.search(
    query_embedding,
    current_search_limit,
)
dense_results = normalize_dense_results_to_unit_interval([...])
if existing is None or result["score"] > existing["score"]:
    aggregated_results[key] = result
```

This is the main dense retrieval method. It searches each FAISS article-variant index, normalizes dense scores, and keeps the best dense score per candidate page across variants.

```
- average_dense_component_results()

for component_results in component_results_sets:
    for result in component_results:
        entry["score"] += result["score"]
    result["score"] /= total_components
```

This method combines the five generated-question DPR searches. A page that appears for multiple generated questions gets an averaged dense score instead of being counted five separate times.

```
- build_faiss_precomputed_components()

precomputed_components.append(
    ("faiss_score", faiss_weight, search_dpr_faiss(...))
)
natural_question_results = [
    search_dpr_faiss(...) for natural_question_embedding in ...
]
avg_results = average_dense_component_results(natural_question_results)
precomputed_components.append(
    ("natural_questions_avg_faiss_score", weight, avg_results)
)
```

This method collects the dense retrieval signals used in the final Stage 1 score. It covers the category+clue DPR query, the average of five generated-question queries, and the concatenated generated-question query.

```
- normalize_results_to_unit_interval() and normalize_dense_results_to_unit_interval()

"raw_score": result["score"],
"score": result["score"] / max_score

"score": (result["score"] - min_score) / score_range
```

These methods put sparse and dense component scores onto comparable scales before weighting. Sparse scores use max-score normalization, while dense scores use min-max normalization.

```
- search_whoosh_weighted_with_searcher()

entry[score_name] = max(entry[score_name], result["score"])
result["score"] = title_weight * result["title_score"]
result["score"] += body_weight * result["body_score"]
result["score"] += component_weight * result[score_name]
```

This is the main Stage 1 scoring method. It merges all component candidate lists by page, fills missing component scores with zero, transfers redirect matches to canonical pages, applies the final weights, and sorts by total weighted score.

- `src/run_test_100.py`
 - `search_questions()`

```

queries = [
    question_query(question, query_mode, include_category)
    for question in questions
]
dense_query_embeddings = get_precomputed_dense_query_embeddings(...)
return run_search_batch(queries, mode=mode, weighted_cole=weighted_cole, ...)

```

This method runs Stage 1 for the 100-question set. It assembles the query text, loads precomputed embeddings, and calls the selected retrieval mode.

– `cross_encoder.query_texts()` and `cross_encoder.document_text()`

```

if query_mode == "natural_questions_avg":
    return natural_questions
return f"Title: {title}\nArticle: {body}"

```

These methods define the actual text sent to the Stage 2 cross-encoder. In the final setting, each generated natural question is paired with candidate article evidence.

– `chunk_text_by_whitespace_tokens()`

```

for start in range(0, len(tokens), chunk_token_limit):
    token_chunk = tokens[start : start + chunk_token_limit]
    chunks.append(" ".join(token_chunk).strip())

```

This method splits long articles into fixed-size chunks. Stage 2 scores chunks instead of full articles so the cross-encoder can focus on local evidence.

– `score_cross_encoder_pairs()`

```

cached_score = load_cached_cross_encoder_score(model_name, text_a, text_b)
predicted_scores = model.predict(uncached_pairs, batch_size=batch_size)
store_cached_cross_encoder_score(model_name, pair[0], pair[1], score_value)

```

This method runs the cross-encoder and caches pair scores. The cache matters because reranking creates many question/chunk pairs for the same candidates.

– `rerank_results_with_cross_encoder()`

```

document_chunks = chunk_text_by_whitespace_tokens(document_text, ...)
pairs.append((query_text, chunk_text))
pair_scores = score_cross_encoder_pairs(pairs, model_name=model_name)
best_score = float(np.mean(query_scores))

```

This method implements the Stage 2 cross-encoder scoring. It compares generated natural questions against article chunks and averages the best evidence score per question.

– `fuse_initial_and_cross_encoder_results()`

```

normalized_initial_scores = min_max_normalize_values(initial_scores)
normalized_cross_scores = min_max_normalize_values(cross_encoder_scores)
fused_score = (
    initial_weight * initial_score
    + cross_encoder_weight * cross_encoder_score
)

```

This method creates the final Stage 2 score. It normalizes Stage 1 and cross-encoder scores inside the top-100 set and then applies the selected fusion weights.

– `build_stage2_searches()`

```

missed_reranked = rerank_searches_with_cross_encoder(...)
missed_fused = fuse_searches_with_cross_encoder(missed_reranked)
store_cached_stage2_ranking(...)

```

This method combines the Stage 1 and cross-encoder scores into the Stage 2 ranking. It also caches the Stage 2 output so later runs and Qwen export do not recompute the same model scores.

– `first_answer_rank()` and `correct_at_k_values()`


```

if normalize_label(result["title"]) in answers:
    return rank
if answer_rank <= k:
    matched_values.add(k)

```

These methods implement title-based answer matching. They determine where the gold page first appears and which Top- k metrics should count the clue as correct.

```

- compute_metrics_from_searches()

answer_rank = first_answer_rank(results, answers)
for k in correct_at_k_values(results, answers, top_k_values):
    correct_counts[k] += 1
return {k: correct_counts[k] / total for k in top_k_values}

```

This method produces the Top- k evaluation values for Stage 1 and Stage 2. It checks normalized title matches between retrieved pages and the gold answer.

- `src/export_qwen_verifier_inputs.py`

```

- candidate_key()

raw_key = json.dumps(..., sort_keys=True, ensure_ascii=False)
return hashlib.sha256(raw_key.encode("utf-8")).hexdigest()

```

This method creates a stable identifier for each exported candidate row. The export file uses these keys to connect Stage 2 candidates with Qwen scoring outputs.

- `src/colab_score_qwen_verifier_inputs.py`

```

- reranker_text() and process_reranker_inputs()

f"<Instruct>: {RERANKER_INSTRUCTION}\n"
f"<Query>: {query}\n"
f"<Document>: {document}"
inputs["input_ids"][index] = prefix_tokens + input_ids + suffix_tokens

```

These methods build the Qwen3-Reranker-4B input format. They wrap each natural question and evidence snippet in the reranker instruction template.

```

- reranker_probabilities()

next_token_logits = model(**encoded).logits[:, -1, :]
yes_logits = next_token_logits[:, yes_id]
no_logits = next_token_logits[:, no_id]
return torch.softmax(yes_no_logits, dim=1)[:, 1].tolist()

```

This method converts the reranker model's yes/no logits into a probability-like relevance score. That score is what Stage 3 uses for final candidate ordering.

```

- score_reranker_row()

reranker_pairs = [
    reranker_text(question, row["evidence"])
    for question in natural_questions
]
reranker_scores.extend(reranker_probabilities(...))
average_reranker_score = sum(reranker_scores) / len(reranker_scores)

```

This method scores one Stage 3 candidate with Qwen3-Reranker-4B. It averages the reranker probabilities across the five generated natural questions for that clue.

```

- compute_topk_metrics()

ranked = sorted(
    rows,
    key=lambda row: (-float(row[score_field]), int(row["candidate_rank"])),
)
top_titles = {normalize_label(row["title"]) for row in ranked[:k]}
if normalized_answer in top_titles:
    correct_counts[k] += 1

```

This method computes the Stage 3 Top- k results after Qwen scoring. It uses the reranker score to sort the exported top-10 candidates for each clue.

The main 100-question evaluation can be run from the project root with:

```
python -m src.run_test_100 --weighted-cole --include-category \
--cross-encoder --cross-encoder-mode chunks_100 \
--cross-encoder-query-mode natural_questions_avg \
--cross-encoder-natural-questions \
data/processed/questions_natural_qwen3_8b.json
```

This command evaluates Stage 1 and Stage 2 on the 100-question set and prints the Top- k metrics. For Stage 3, the local code first exports the top 10 Stage 2 candidates:

```
python -m src.export_qwen_verifier_inputs \
--output data/processed/qwen_verifier_inputs.jsonl --top-docs 10
```

The exported JSONL file is then scored in Google Colab with the Qwen scoring script:

```
python colab_score_qwen_verifier_inputs.py \
--input qwen_verifier_inputs.jsonl \
--output qwen_verifier_scores.jsonl \
--chat-model-name Qwen/Qwen3-4B \
--reranker-model-name Qwen/Qwen3-Reranker-4B \
--device cuda
```

The Stage 1/2 command prints Top- k metrics locally, and the Qwen scoring script prints the Stage 3 Top- k metrics after scoring.

4.1 Implementation Workflow (Coding with LLMs)

We used LLM tools, including Codex, Claude, and ChatGPT, throughout the implementation process as coding and research-support assistants. They helped us scaffold new components, generate code templates, propose project directory structures, debug basic errors, refine index creation code, suggest model choices, and compare alternative retrieval designs such as cross-encoder reranking. We also used them to better understand related literature and theoretical mechanisms through simplified examples, which helped us decide which components were worth implementing in our retrieval pipeline. In addition, they provided useful suggestions for organizing the codebase, planning experimental branches, structuring the paper, and naming sections.

However, these tools were not always reliable: some outputs were overly complex, did not align with our previous experimental results, misunderstood the code flow, or failed to capture the main research question. Therefore, we did not use generated code or writing suggestions blindly. We carefully reviewed, documented, corrected, and adapted the outputs to fit the actual behavior of the Wikipedia data and our pipeline. Overall, LLM assistance reduced implementation time and improved development speed, but correctness, coherence, and final design decisions still depended on manual inspection, debugging, and integration.

4.2 Compute Environment

We developed and ran the project across two environments: a local MacBook setup and Google Colab. The local machine was used for the main retrieval codebase, indexing, weight tuning, and most iterative debugging. Google Colab was used for larger model runs, especially Qwen-based question generation, summary generation, and reranking experiments.

Table 2: Main system-stage comparison on the 100-question set. P@1 measures whether the top-ranked page title is correct, while Top-5 shows whether the correct page appears anywhere in the first five results.

System stage	P@1	Top-5
Sparse body-only baseline, no category	0.21	0.42
Sparse body-only baseline + category	0.22	0.46
Stage 1: weighted hybrid retrieval + category	0.31	0.50
Cross-encoder only	0.40	0.62
Stage 2: Stage 1 + cross-encoder	0.41	0.61
Qwen chat verifier	0.04	0.29
Stage 3: Qwen3-Reranker-4B	0.47	0.58

5 Evaluation Setup

As mentioned earlier, we evaluate on the same 100 clues used throughout development, over the final indexed collection of 152,042 non-redirect Wikipedia articles.

The final evaluated system uses the category together with the clue text as the initial query. It first produces a global candidate ranking with the Stage 1 weighted hybrid retriever described in Section 3, then reranks the top 100 candidate pages in Stage 2 by combining the Stage 1 score with the cross-encoder score averaged across five generated natural questions, and finally reranks the top 10 candidates from Stage 2 with Qwen3-Reranker-4B (Section 4; `file:src/run_test_100.py, method: evaluate_questions()` `file:src/colab_score_qwen_verifier_inputs.py, method: compute_topk_metrics()`).

We evaluate performance using normalized title matching between the top-ranked predicted page and the gold answer (Section 4; `file:src/run_test_100.py, method: first_answer_rank()` `file:src/run_test_100.py, method: correct_at_k_values()`). The primary metric is Precision@1 (Top-1 accuracy), because the system must commit to a single final answer. We also report Top-5 as a supporting metric to indicate whether the correct page appears near the top of the ranking before the final decision.

We directly tuned retrieval weights and selected configurations on the same set of 100 questions used for evaluation. We did not use a separate training or development split because the dataset is small.

6 Results and Analysis

6.1 Main Results

Table 2 presents the main system-stage comparison (Section 4; `file:src/run_test_100.py, method: compute_metrics_from_searches()` `file:src/colab_score_qwen_verifier_inputs.py, method: compute_topk_metrics()`). The largest early gains come from better retrieval. The sparse-only baselines reach P@1 scores of 0.21 and 0.22, while the full weighted hybrid configuration reaches 0.31. This shows that most of the early improvement comes from stronger candidate generation rather than from later reranking alone.

The cross-encoder then improves answer selection within the retrieved set. Using only the cross-encoder ranking over the top 100 candidates yields a P@1 score of 0.40, and the full Stage 2 ranking reaches 0.41. This suggests that Stage 1 often retrieves the correct page, but because it primarily matches terms rather than meaning, it may not rank that page first.

The final Qwen reranker produces the best Top-1 result, improving the P@1 score from 0.41 to 0.47. In absolute terms, the final system returns the correct page at rank 1 for 47 out of 100 clues. This is still a reranking step, similar to Stage 2, but Qwen3-Reranker-4B is a larger model and can make more accurate distinctions among the strongest candidates. However, this improvement should be interpreted carefully:

Table 3: Top-1 correct counts by super-category. The examples column shows representative original categories, N is the number of questions in the group, and each stage column reports correct top-ranked answers with the group percentage in parentheses.

Super-category	Examples	N	Stage 1	Stage 2	Stage 3
People	Golden Globe Winners; '80s No.1 Hitmakers	33	15 (45%)	18 (55%)	19 (58%)
Geography	African Cities; Capital City Churches	17	4 (24%)	5 (29%)	8 (47%)
History	1920s News Flash!; Notes from the Campaign Trail	25	4 (16%)	8 (32%)	9 (36%)
Organizations	Name the Parent Company; Service Organizations	8	4 (50%)	2 (25%)	4 (50%)
Media	Newspapers	5	2 (40%)	2 (40%)	3 (60%)
Language	Complete Dom-ination	3	0 (0%)	0 (0%)	1 (33%)
Others	Greek Food & Drink; Potpourri	9	2 (22%)	6 (67%)	3 (33%)
Total		100	31 (31%)	41 (41%)	47 (47%)

Qwen3-Reranker-4B only processes the top 10 candidates from Stage 2. It improves final answer selection within that set but does not improve recall beyond what earlier stages retrieved.

The Qwen reranker also lowers Top-5 from 0.61 to 0.58, even though P@1 increases. This indicates that it is better at placing a single correct answer at the top but does not preserve as many correct answers within the top five. Since the primary goal is to return one final answer, the improvement in P@1 is more significant.

One negative result is also noteworthy. The Qwen chat verifier performs poorly, achieving only a P@1 score of 0.04. A yes/no chat probability is thus not an effective final ranking signal for this task, even when the underlying model is larger than the cross-encoder. This setup also does not use a dedicated reranking model or require the model to answer the clue directly. While more explicit reasoning might improve chat-based performance, it would also slow retrieval, which is undesirable for an information retrieval system.

6.2 Grouped Category Analysis

To make the category-level behavior interpretable, we group the 30 original clue categories into 7 super-categories. Entertainment categories whose answers are mainly people are included under *People*, and business categories are grouped with organizations. Table 3 shows Top-1 counts at each stage.

The grouped analysis shows that the system is strongest on entity-centered groups. *People*, *Media*, and *Organizations* often contain names, roles, outlets, companies, or institution descriptions that appear directly in Wikipedia article text. These clues give the retriever concrete terms to match and give the rerankers explicit evidence to compare.

History and *Others* benefit most from Stage 2. These clues are often written as indirect descriptions rather than direct title matches, so a cross-encoder can help by comparing the rewritten question with local article evidence. This is where semantic matching is most useful: the correct page may already be present, but Stage 1 may rank a more term-similar distractor above it.

Stage 3 helps most when the top candidates are close but the evidence snippet contains a direct clue. This explains the improvement for *Geography* and *Organizations*: location pages, company pages, subsidiaries, and institutions can all look similar to earlier rankers, but the larger reranker is better at choosing among a small set of strong candidates.

The drops also show a limitation of late reranking. *Others* becomes worse at Stage 3, suggesting that the final reranker is less stable for broad, mixed, or comparative clues where no single evidence sentence clearly identifies the answer. *Language* remains difficult because wordplay categories often have weak lexical overlap with the answer page, so even stronger rerankers have little reliable evidence to use.

6.3 What Did Not Work

There were many explored approaches that were not included in the final system. We created a custom pipeline which executed tokenization and normalization. We included explicit lowercasing, stop-word removal, punctuation stripping, and token splitting in this pipeline. This was tested, but Whoosh’s default analyzer performed better in the final sparse index. Lead-only indexes based on the first sentence, first two sentences, or first paragraph all underperformed full-body retrieval when used as standalone signals. Title-only and redirect-only retrieval also failed as standalone ranking methods. Redirect-only retrieval was intended as an alias signal, where matches on redirect titles would boost canonical pages, but this did not improve the final weighted configuration.

We also explored clue paraphrasing for sparse retrieval, but it did not improve the performance significantly. The paraphrases mostly preserved the key nouns and proper nouns and only modified the surface wording which resulted in minimal additional lexical evidence for BM25-style matching. Also, several implemented components like body summary dense indexes and category semantic variants remained at zero weight in the final configuration because they did not provide any improvement.

7 Conclusion

Wikipedia clue answering benefits from a staged retrieval design. Stage 1 is critical for broad candidate generation, Stage 2 improves semantic ranking over the top 100 candidates, and Stage 3 is effective only after the candidate set has been reduced to strong alternatives.

The main conclusion is that retrieval quality is the most important factor. Larger rerankers are beneficial once the correct page has already been retrieved, but they cannot recover answers that are missing from the candidate set. Most remaining errors come from retrieval mistakes, ranking confusion between similar pages, and wordplay clues that have little direct overlap with the target page.

References

- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- Qwen Team. Qwen3 technical report, 2025.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR*, abs/1908.10084, 2019. URL <http://arxiv.org/abs/1908.10084>.

A Prompt and Text Templates

Figure 2: Prompt and text templates used by the pipeline. The table lists each component, where it is used, and the exact input format or prompt pattern used for retrieval, reranking, and failed auxiliary experiments.

Component	Where used	Template or format
Sparse query text	Stage 1 retrieval	Category and clue concatenated into one query string: {category}{clue}.
Natural-question generation with Qwen3-8B	Question rewriting before Stage 1 and later reranking	System: "You rewrite clue-style prompts into natural search and QA questions. Return valid JSON only." User: "Write exactly 5 natural-language questions from the category and clue ... Use only the information in the category and clue ... Do not reveal the answer ... Return exactly this JSON shape and nothing else: {"questions":["...", "...", "...", "...", "..."]}." Input fields: Category: {category} and Clue: {clue}.
Cross-encoder query text	Stage 2 reranking	Each of the five generated natural questions is used directly as the query text.
Cross-encoder document text	Stage 2 reranking	Title: {title}\nArticle: {cleaned_article_chunk}
Body-summary prompt	Body-summary dense-index experiment using Qwen-generated summaries	System: "You summarize Wikipedia-style articles for retrieval. Preserve unique facts, names, dates, places, aliases, and distinguishing details. Return valid JSON only." User: "Write one concise factual summary for dense retrieval. Use 3 to 5 sentences ... Return exactly this JSON shape and nothing else: {"summary": "..."}." Input fields: Title: {title} and Article text: {article_text}.
Chat-verifier prompt	Failed Qwen3-4B chat-model verifier	Question: {natural_question}\nArticle: {evidence}\n\n=====\n\nDoes this article contain the answer to the question? Answer with exactly one word: Yes or No.
Reranker content text	Final Qwen3-Reranker-4B stage	<Instruct>: {instruction}\n<Query>: {natural_question}\n<Document>: {evidence}
Reranker instruction	Final Qwen3-Reranker-4B stage	"Given a clue rewritten as a natural question, retrieve articles that contain enough information to identify the answer." The model is further wrapped with a system prefix that restricts the decision to "yes" or "no."